

Aprendizaje Automático

Introducción a las prácticas

Manuel Pegalajar Cuéllar¹

¹Departamento de Ciencias de la Computación e Inteligencia Artificial. Universidad de Granada

Contenido

Parte I: Información general de las prácticas

- 1 Profesor/es de prácticas
- 2 Objetivos y Metodología
- 3 Materiales de prácticas
- 4 Primer uso de Colab y bibliotecas asociadas
- 5 Cómo realizar la entrega de prácticas

Parte II: Herramientas de prácticas

- 6 La biblioteca NumPy
- 7 Visualización de datos con Matplotlib
- 8 Lectura de datos con Pandas
- 9 Scikit-Learn

Parte I

Información general de las prácticas



Contenido

- 1 Profesor/es de prácticas
- 2 Objetivos y Metodología
- 3 Materiales de prácticas
- 4 Primer uso de Colab y bibliotecas asociadas
- 5 Cómo realizar la entrega de prácticas

- Manuel Pegalajar Cuéllar (grupos de los lunes y los miércoles)
- E-Mail: manupc@ugr.es
- URL: <https://www.ugr.es/personal/manuel-pegalajar-cuellar>
- Localización: Despacho D19. 4a. planta de la ETSIIT.
- Tutorías:

Segundo semestre

DÍA	HORARIO	LUGAR
Lunes	10:30 - 12:30	D19 (Etsiit)
Lunes	15:30 - 17:30	D19 (Etsiit)
Martes	10:30 - 12:30	D19 (Etsiit)

Figura 1: Horario de tutorías personalizadas.

Concertar cita previa por e-mail.

- Marios Kountouris (grupo de los viernes)
- E-Mail: mariosk@ugr.es
- URL: <https://www.ugr.es/personal/marios-kountouris>

Concertar cita previa por e-mail.

Contenido

- 1 Profesor/es de prácticas
- 2 Objetivos y Metodología**
- 3 Materiales de prácticas
- 4 Primer uso de Colab y bibliotecas asociadas
- 5 Cómo realizar la entrega de prácticas

Objetivos de las prácticas

- Llevar a la práctica los conceptos de teoría.
- Que aprendáis para después aprobar.

¿Cómo?

Planificación de las prácticas:

- P0 Introducción a Python, Numpy y Sk-Learn. (*no evaluable*)
- P1 Práctica de aprendizaje supervisado.
- P2 Práctica de aprendizaje no supervisado.
- P3 Práctica de aprendizaje automático (más) avanzado.

Proyecto **Proyecto práctico** **opcional** a desarrollar por el estudiante.

- Exposiciones de repaso por parte del profesor.
- Seminarios de introducción a las prácticas.
- Resolución de ejercicios en clase y trabajo autónomo.
- Corrección/feedback sobre las entregas *ASAP*.

La asistencia a prácticas **no es obligatoria**, pero sí **recomendable**.

Además...

- Quiero buen ambiente en clase.
- **Nadie nace sabiendo**. Todos aprendemos (yo incluido). Solicita ayuda cuando la necesitéis.
- Dadme feedback sobre vuestras necesidades de aprendizaje.

Planificación inicial estimada:

P0 17/Feb al 7/Mar.

P1 10/Mar al 30/Mar (**entrega**).

P2 31/Mar al 27/Abr (**entrega**).

P3 28/Abr al 25/May (**entrega**).

Proyecto **Trabajo por grupos de 2** a lo largo del semestre. Plazo límite: 30/May.

Entregas: Con cada trabajo/práctica se entrega un **Notebook** que integra código, resultados y discusión.

- La **discusión** de los resultados, y el **análisis** y **descripción** del trabajo realizado, resultan fundamentales (*es importante que los resultados sean correctos, pero todavía lo es más demostrar que se entiende lo que se está haciendo*).

Evaluación de las prácticas

La evaluación de las prácticas será la siguiente:

P0 No evaluable.

P1 Máximo de 10 puntos.

P2 Máximo de 10 puntos.

P3 Máximo de 10 puntos.

Proyecto **OPCIONAL. EN GRUPOS DE 2.** Máximo de 10 puntos.

La **calificación de prácticas** se obtiene mediante la fórmula:

$$PR = 2 \frac{P1 + P2 + P3 + Proyecto}{10}$$

La **calificación FINAL de prácticas** se encuentra en el rango $[0, 8]$.

¿Por qué un proyecto práctico opcional?

El aprendizaje automático es muy extenso, y avanza a ritmos vertiginosos.

Posiblemente, lo que enseñamos hoy en el aula quede obsoleto en pocos años.

El **aprendizaje a lo largo de la vida** resultará esencial para mantenerse actualizado.

El objetivo principal del proyecto opcional es, precisamente, favorecer la **capacidad de auto-aprendizaje** para ser capaz de mantenerse actualizado y estar preparado para aprender cuestiones novedosas.

Para ello, el proyecto tratará sobre **temáticas que no se hayan tratado en teoría** o, en su defecto, **se hayan visto en menor profundidad**.

Objetivo secundario: DISFRUTAR del aprendizaje automático aprendien-

Contenido

- 1 Profesor/es de prácticas
- 2 Objetivos y Metodología
- 3 Materiales de prácticas**
- 4 Primer uso de Colab y bibliotecas asociadas
- 5 Cómo realizar la entrega de prácticas

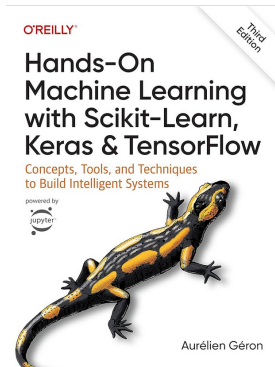
Materiales necesarios

Trabajaremos fundamentalmente con:

- **Google Colab**. Herramienta online de Google para desarrollo de scripts Python. Accesible mediante la cuenta @go.ugr.es.
- **NumPy**. Biblioteca fundamental de cálculo numérico y científico en Python.
- **Matplotlib**. Biblioteca de visualización y generación de gráficas.
- **Pandas**. Biblioteca fundamental en Ciencia de Datos.
- **Scikit-Learn**. Biblioteca Python de aprendizaje automático robusta y bien testada, construida sobre las bibliotecas **NumPy** y **SciPy**.
- **TensorFlow/Keras**. Biblioteca Python de Google para desarrollo de modelos avanzados de aprendizaje automático y Deep Learning.

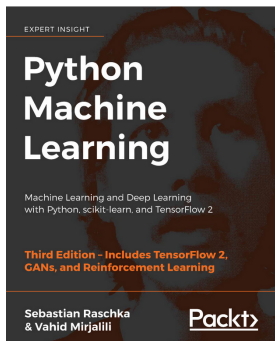
Bibliografía fundamental

Géron, Aurélien. Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media, 2022.



Notebooks disponibles online:
<https://github.com/ageron/handson-ml3>

Sebastian Raschka, Vahid Mirjalili, Python Machine Learning. Packt. 2019



Notebooks disponibles online:

<https://github.com/rasbt/python-machine-learning-book-3rd-edition>

Contenido

- 1 Profesor/es de prácticas
- 2 Objetivos y Metodología
- 3 Materiales de prácticas
- 4 Primer uso de Colab y bibliotecas asociadas**
- 5 Cómo realizar la entrega de prácticas

Uso de Google Colab en las prácticas

Accesible online desde <https://colab.research.google.com/>

- Con cada práctica entregaréis un **fichero .ipynb** exportado desde Colab.
- En dicho fichero se incorpora **código, resultados, discusión y análisis** de los mismos. Actúa como un *“todo en uno”*; es decir, un fichero de Colab integra implementación y documento de memoria de prácticas.
- El uso de Colab no significa que podáis realizar una memoria de menor calidad o menor grado de detalle que la que redactaríais en un PDF independiente.

Se recomienda revisar con atención la documentación oficial:

<https://colab.research.google.com/notebooks/intro.ipynb>

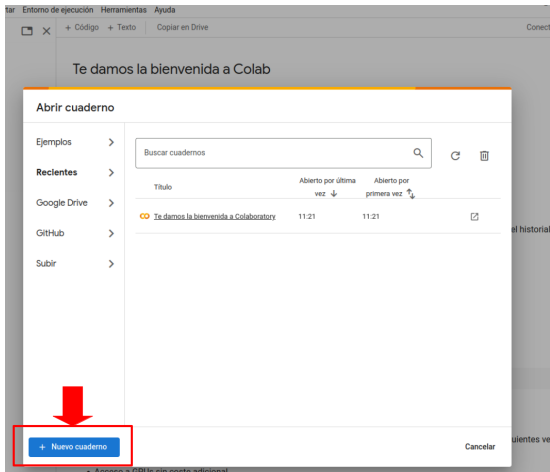
Pasos para usar Google Colab

1. Accedemos a <https://colab.research.google.com/> e iniciamos sesión con nuestra cuenta **@go.ugr.es**.



Pasos para usar Google Colab

2. Acto seguido, se nos permite abrir un cuaderno *Notebook* desde distintas fuentes, o bien crear uno nuevo (el cual se almacenará en nuestro espacio de *GDive*).



Pasos para usar Google Colab

3. El cuaderno ya está creado en nuestro GDrive, con nombre **untitled?.ipynb**. El nombre se puede cambiar haciendo clic sobre él.

Nombre del fichero. Extensión .ipynb (interactive **py**thon **no**tebook)

The screenshot shows the Google Colab web interface. The browser address bar displays the URL: `colab.research.google.com/ide/t/n2w1oKbQFw8QXfGKcmW1s_w03ygzf_Qc`. The notebook title is `Untitled1.ipynb`, which is highlighted with a red box and an arrow pointing to the text "Nombre del fichero. Extensión .ipynb (interactive **py**thon **no**tebook)". Below the title bar, the menu includes "Archivo", "Editar", "Ver", "Insertar", "Entorno de ejecución", "Herramientas", and "Ayuda". The "Código" and "Texto" tabs are highlighted with a red box and an arrow pointing to the text "Crear nueva celda de texto". The main workspace contains a single code cell, which is highlighted with a red box and an arrow pointing to the text "Celda actual". On the right side of the workspace, the "Eliminar celda" button (trash icon) is highlighted with a red box and an arrow pointing to the text "Eliminar celda". At the bottom left of the workspace, the "Crear nueva celda de código" button (plus icon) is highlighted with a red box and an arrow pointing to the text "Crear nueva celda de código".

Primer contacto con Colab

- 1 Copia el fichero [Intro_Notebook.ipynb](#) y el fichero [UGR-Logo.png](#) a tu cuenta de GDrive, dentro de la carpeta [Colab Notebooks](#). También puedes **subir** el fichero desde el menú [Archivo-subir cuaderno](#).
- 2 Abre el cuaderno desde Colab.

Te interesa practicar (tienes 5' para hacerlo):

- Creación de celdas de texto con distintos formatos, celdas de código, borrarlas, ejecutarlas...
- Intercalar celdas de texto y de código Python.
- Conocer cómo ejecutar una única celda, todas las celdas y reiniciar la sesión para ejecutar todo.
- Familiarizarte con el concepto de sesión activa. **Presta atención a las celdas 8 y 9 de código: Son fuentes de muchos errores de programación con Notebooks.**
- Te conviene familiarizarte con las macros (sobre todo de creación/borrado/ejecución de celdas).

Contenido

- 1 Profesor/es de prácticas
- 2 Objetivos y Metodología
- 3 Materiales de prácticas
- 4 Primer uso de Colab y bibliotecas asociadas
- 5 **Cómo realizar la entrega de prácticas**

Normas de entrega:

- Se entregará, a través de PRADO, un único **Notebook** por cada práctica (extensión **.ipynb**).
- El Notebook será autocontenido y tendrá celdas de texto y de código.
- Se deberá organizar en **secciones**.
- La primera celda deberá contener:
 - Un título de la práctica.
 - El número de práctica, la asignatura y el curso académico.
 - **Nombre y apellidos del autor.**
- En las celdas de texto se deberá explicar la formulación del problema, cómo se ha abordado, una descripción y análisis de resultados, discusión de los mismos y conclusiones finales.

No hay informe/memoria de prácticas en formato PDF. Dicho informe se incluye dentro del notebook como celdas de texto. Por tanto, el contenido debe cumplir un objetivo fundamental, que consiste en explicar:

- qué se ha hecho,
- cómo se ha hecho lo que se ha hecho,
- porqué se ha hecho así (motivación),
- qué resultados se han obtenido, y
- qué conclusiones podemos obtener a partir de los mismos.

Un informe debe ser claro, conciso y bien estructurado. Si no hay informe/discusión, es como no entregar nada (con su correspondiente calificación de 0 puntos).

Cuadernos Colab en las entregas de prácticas

Estructura básica del informe (que puede/debería intercalarse con celdas de código):

- 1 **Portada**: la primera celda.
- 2 **Índice**: que permita comprender la estructura general de los contenidos y también localizarlos con facilidad.
- 3 **Introducción**: se describe el problema a resolver (¿qué queremos hacer?), la motivación (¿por qué es relevante hacerlo?), y los objetivos (¿qué objetivos concretos vamos a abordar de cara a resolver el problema?).
- 4 **Fundamentos teóricos**: se presentan los conceptos necesarios para comprender el trabajo. **No se trata de repetir mecánicamente lo que se indica en teoría/prácticas, sino en demostrar que se comprenden los conceptos fundamentales.**
- 5 **Estado del Arte**: se presenta qué se ha hecho en el campo con anterioridad, y cuáles son los mejores métodos en la actualidad.

Cuadernos Colab en las entregas de prácticas

Estructura básica del informe (continuación):

- 6 **Métodos**: descripción detallada de los métodos empleados y/o propuestos.
- 7 **Experimentos**:
 - Descripción del conjunto de datos empleado.
 - Protocolo de validación experimental.
 - Métricas empleadas (para entrenamiento y en validación).
 - Experimentos realizados.
 - Resultados obtenidos.
 - Discusión sobre los mismos.
- 8 **Conclusiones**: **No se trata de una conclusión subjetiva por parte del que crea el informe.** Se presenta, de modo breve y a modo de resumen, las principales conclusiones del trabajo realizado.
- 9 **Bibliografía**: Contiene todas las referencias manejadas por el estudiante a la hora de realizar el trabajo. **Los apuntes o diapositivas del profesor no son bibliografía.**

Algunas recomendaciones:

- Se deben **discutir y analizar los resultados** obtenidos, tanto a nivel cualitativo como cuantitativo. No vale decir *“Podemos ver el resultado durante la ejecución del programa”*.
- **MUY IMPORTANTE:** incluir información que ponga en valor el trabajo realizado:
 - ventajas e inconvenientes de los métodos empleados, problemas encontrados a la hora de realizar la práctica,
 - descripción de experimentos fallidos realizados,
 - motivos que nos llevan a usar unos valores para los parámetros en lugar de otros, etc.
- Las figuras y tablas deben verse correctamente (una figura borrosa o con fuente muy pequeña no cumple su función). En las tablas, limitad el número de decimales a 2 o 3.

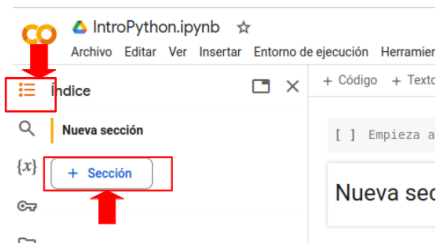
Algunas recomendaciones:

- Usad figuras con diagramas para apoyaros en las explicaciones. Un texto se entiende mejor si hay una figura o gráfica asociada.
- Integrad las fórmulas y las ecuaciones dentro del mismo documento.
- Usad LaTeX integrado en el cuaderno.

En resumen, tenemos que hacernos la siguiente pregunta: *Si nosotros no hubiésemos escrito esta memoria, al leerla, ¿comprenderíamos el trabajo desarrollado? ¿Seríamos capaces de valorar el trabajo realizado y los conocimientos de quien lo ha hecho?*

Creación de secciones en cuadernos Colab

Se pueden crear índices en Google Colab de forma sencilla y visual:



Creación de secciones en cuadernos Colab

Se pueden crear subsecciones de índices en Google Colab. También se puede hacer de forma manual en una celda de texto incluyendo símbolos # de forma sencilla y visual:

- # sección (1)
- ## subsección (1.1)
- ### subsubsección (1.1.1)
- ### subsubsección (1.1.2)
- ## subsección (1.2)
- # sección (2)
- ## subsección (2.1)
- ...

Creación de secciones en cuadernos Colab

Un ejemplo:

- ▼ Introducción

[] Empieza a programar o a crear código con IA.

- Objetivos

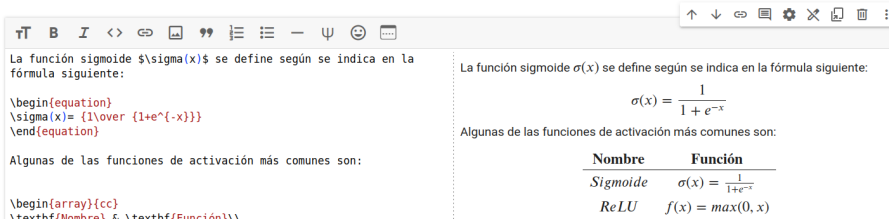
Integración de LaTeX en cuadernos Colab

Tutoriales sobre LaTeX:

- OverLeaf: <https://www.overleaf.com/learn/latex/Tutorials>
- Otro tutorial: <https://latex-tutorial.com/tutorials/>

Ejemplo: Fichero [LatexColab.ipynb](#)

✓ Ejemplo de uso de ecuaciones LaTeX



The image shows a LaTeX Colab interface. On the left is a code editor with a toolbar containing icons for undo, redo, bold, italic, font color, background color, link, unlink, list, unlist, indent, unindent, subscript, superscript, text color, and a smiley face. The code in the editor is as follows:

```
La función sigmoide  $\sigma(x)$  se define según se indica en la fórmula siguiente:
```

```
\begin{equation}
\sigma(x) = \frac{1}{1 + e^{-x}}
\end{equation}
```

Algunas de las funciones de activación más comunes son:

```
\begin{array}{cc}
\textbf{Nombre} & \textbf{Función} \\
\hline
Sigmoide & \sigma(x) = \frac{1}{1 + e^{-x}} \\
ReLU & f(x) = \max(0, x)
\end{array}
```

On the right is the rendered output. It shows the text "La función sigmoide $\sigma(x)$ se define según se indica en la fórmula siguiente:" followed by the rendered equation
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$
. Below this, it says "Algunas de las funciones de activación más comunes son:" followed by a table:

Nombre	Función
Sigmoide	$\sigma(x) = \frac{1}{1 + e^{-x}}$
ReLU	$f(x) = \max(0, x)$

Un ejemplo completo

Ejemplo de cuaderno: Fichero [IntroPython.ipynb](#).

- Copiad el cuaderno [IntroPython.ipynb](#) a vuestro GDrive, en la carpeta dedicada a Notebooks de Colab.
- Se trata del primer tutorial de la asignatura, dedicado a la programación básica utilizando lenguaje Python.
- Seguid sus instrucciones para aprender a programar en Python a nivel introductorio.
- Se puede utilizar también para ver cómo usar diferentes estructuras (imágenes, ecuaciones, secciones...)

Parte II

Herramientas de prácticas

6 La biblioteca NumPy

7 Visualización de datos con Matplotlib

8 Lectura de datos con Pandas

9 Scikit-Learn

NumPy

NumPy es un referente en Python para la realización de cálculos matemáticos.

Introduce el tipo de dato array/array multidimensional `ndarray`.



Referencia completa a la biblioteca:

<https://numpy.org/doc/stable/reference/index.html>

Uso en Python: Normalmente importándola con alias.

```
1 import numpy as np
```

Creación de arrays

Normalmente como:

- ```
array= np.array(lista)
```

Para crear un array a partir de una lista (o array multidimensional a partir de lista de listas)

- ```
array= np.zeros((dim1, dim2, ..., dimn))
```

Para crear un array de n dimensiones, cada dimensión i con tamaño **dimi**, inicializado a 0.

- ```
array= np.ones((dim1, dim2, ...))
```

Para crear un array de  $n$  dimensiones, cada dimensión  $i$  con tamaño **dimi**, inicializado a 1.

La variable **array** se crea con tipo de dato **ndarray** de NumPy.

# Creación de arrays

Normalmente como:

- `array= np.empty(dim1, dim2, ...)`

Para crear un array de  $n$  dimensiones, cada dimensión  $i$  con tamaño **dimi**, inicializado con basura.

- `array= np.arange(start, end, step)`

Para crear un array de forma similar a la función **range** de Python.

- `array= np.linspace(start, end, n)`

Para crear un array 1D conteniendo **n** valores entre **start** y **end** (**inclusive**) distribuidos uniformemente.

Parámetros opcionales: Dependen de las funciones.

**dtype**: Permite establecer el tipo de dato de las componentes del array.

# Atributos accesibles del tipo *ndarray*

Hay bastantes. Los más comunes son:

- `array.size`

El número total de componentes en el array.

- `array.ndim`

El número de dimensiones que tiene el array.

- `array.shape`

Tupla de `array.ndim` componentes, donde cada componente indica el número de elementos posibles del array en dicha dimensión.

- `array.dtype`

El tipo de dato de las componentes del array.

# Tipos de datos de las componentes de un array

Son los posibles valores para **dtype**. Hay muchos, pero destacamos los más comunes:

- Todos los tipos básicos de Python (int, float, complex, str, object)
- Enteros con diversa precisión: np.int16, np.int8, np.int32...
- Enteros sin signo con diversa precisión: np.uint16, np.uint8, np.uint32...
- Reales con diversa precisión: np.float16, np.float32, np.float64...
- Complejos con diversa precisión: np.complex64, np.complex128, ...
- etc.

# Ejemplos de inicialización

Ejemplos de inicialización y acceso a atributos de un objeto **ndarray**:

```
1 import numpy as np
2
3 a= np.zeros(10, dtype=int) # Array 1D de 10 0's int
4 a= np.linspace(0, 1, 11) # [0, 0.1, 0.2, ..., 0.9, 1]
5 a= np.ones((10, 10), dtype=float) # Array 2D de 10x10
6 a= np.empty(5, dtype=complex) # array vacío de 5 elms
7 a= np.ones((10, 5), dtype=float) # Array 2D de 10x5
8
9 print(a.size) # 50= 10*5
10 print(a.ndim) # 2
11 print(a.shape, a.shape[0], a.shape[1]) # (10, 5) , 10, 5
12 print(a.dtype) # np.float64
13
```

# Indexación y acceso a componentes

Hay varias formas de acceder a las componentes de un array:

a) Todos los mecanismos de indexación posibles con listas.

```
1 import numpy as np
2
3 a= np.arange(5) # [0, 1, 2, 3, 4]
4
5 print(a[1], a[-1]) # 1, 4
6 print(a[2:]) # 2, 3, 4
7
```

# Indexación y acceso a componentes

Hay varias formas de acceder a las componentes de un array:

b) El operador `:` y la `,`

```
1 import numpy as np
2
3 a= np.array([[1, 2, 3], [4, 5, 6]])
4
5 print(a[0]) # [1, 2, 3]
6 print(a[0][0]) # 1
7 print(a[0,0]) # 1
8 print(a[0, 1:]) # [2, 3]
9 print(a[0, :]) # [1, 2, 3]
10 print(a[:, -1]) # [3, 6]
11
```

# Indexación y acceso a componentes

Hay varias formas de acceder a las componentes de un array:

- c) Utilizando acceso a componentes mediante tupla de índices.
- d) Indexación booleana

```
1 import numpy as np
2
3 a= np.array([[1, 2, 3], [4, 5, 6]])
4 b= np.array([4, 6, 1, 3, 6, 8])
5
6 # Tupla de indices
7 print(a[:, (2, 0)]) # [[3, 1],[6, 4]]
8
9 # Indexacion booleana
10 print(a[a>3]) # [[3, 1],[6, 4]]
11 print(b[b>3]) # [4, 6, 6, 8]
```

# Reorganización de componentes y dimensiones

Hay varias formas de modificar la disposición de las componentes de un array:

- Cambio de la forma (*shape*) de un array con **reshape**.
- Inserción de nueva dimensión con **newaxis**.
- Intercambio de posiciones de dimensiones con **swapaxes**.
- Eliminación de dimensiones de tamaño 1 con **squeeze**.

```
1 import numpy as np
2
3 a= np.array([[1, 2, 3, 4, 5, 6]])
4
5 # reshape
6 print(a.reshape(2, 3)) # [[1, 2, 3], [4, 5, 6]]
7 print(a.reshape(-1, 3)) # [[1, 2, 3], [4, 5, 6]]
8
9 a= a.reshape(1, -1) # [[1],[2],[3],[4],[5],[6]]
10
11 # squeeze
12 a= a.squeeze() # [1, 2, 3, 4, 5, 6]
13
```

# Por dónde empezar con NumPy

La biblioteca NumPy es enorme. Se necesitaría todo un curso para aprender a usar la biblioteca al completo y de forma eficiente. Algunas funcionalidades:

- Operaciones aritméticas elemento a elemento:  $+$ ,  $-$ ,  $*$ ,  $**$ ,  $/$ ,  $\%$ , ...
- Funciones matemáticas: `sqrt`, `pow`, `exp`, `sin`, `tan`, ... (Todas las de `math`).
- Cambio de forma y copia: `reshape` (manteniendo número de componentes), `concatenate`, `hstack`, `vstack`, `copy`, ...
- Funciones de matrices (ndarray 2D): `@`, `dot`, `cross`, `T`, `linalg.det`, `linalg.inv`, ...
- Estadística: `mean`, `std`, `var`, `sum`, `prod`, `cumsum`, `min`, `max`, `argmin`, `argmax`, ...
- Constantes y valores universales: `pi`, `inf`, `-inf`, `nan`, `e`, ...
- Valores aleatorios: `random.seed`, `random.rand`, `random.randn`, `random.randint`, `random.permutation`...

**Cuaderno de iniciación a NumPy:** [IntroNumpy.ipynb](#)

6 La biblioteca NumPy

7 Visualización de datos con Matplotlib

8 Lectura de datos con Pandas

9 Scikit-Learn

## La visualización de datos:

- Permite analizar el comportamiento de la información que tenemos sobre un problema a resolver.
- Permite analizar los resultados de los modelos de IA que generemos para resolverlo.
- Imprescindible para el Análisis Exploratorio de Datos.
- Existen multitud de alternativas, genéricas y especializadas, para mostrar gráficas de gran calidad.



**MatPlotLib** es una de las bibliotecas más conocidas para visualización de datos. Sus capacidades son similares a las de software de pago o tradicionalmente usado para cálculo científico (Matlab, Octave).

Realizar gráficas con la biblioteca es sencillo.

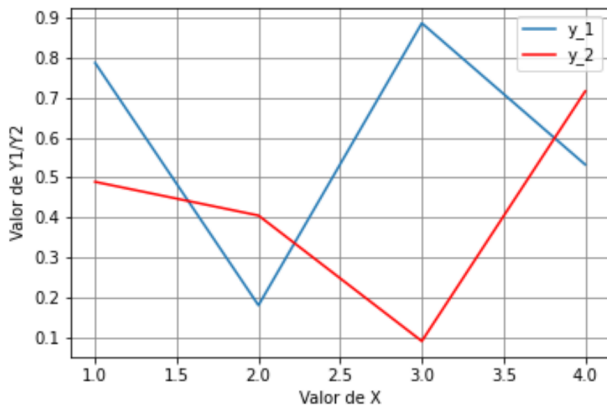
Realizar una visualización atractiva es más complejo, requiere amplios conocimientos de la biblioteca. Referencia completa a la biblioteca matplotlib:

<https://matplotlib.org/stable/index.html>

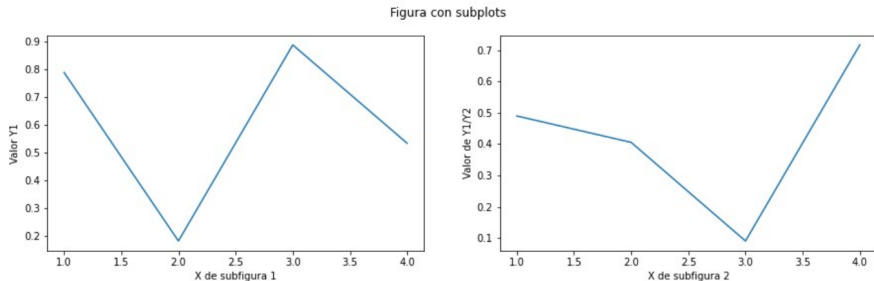
Uso en Python: Normalmente importando sub-paquetes con alias.

```
1 import matplotlib.pyplot as plt
```

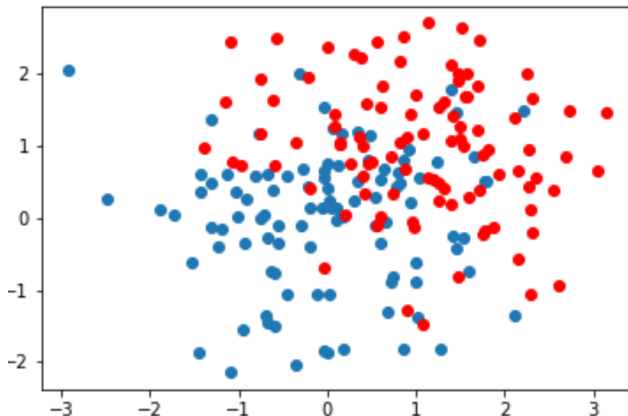
## Gráficos de líneas:



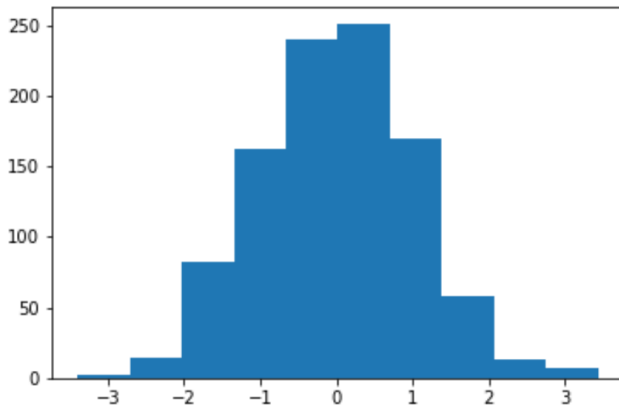
## Múltiples gráficas por figura (subplots):



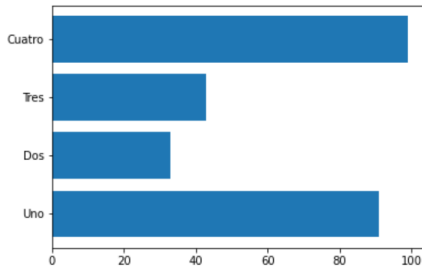
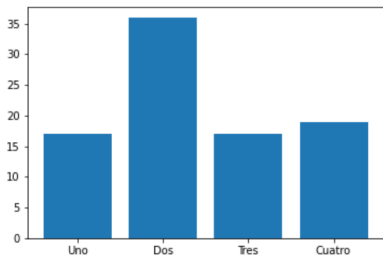
## Gráficos de dispersión (scatter plots):



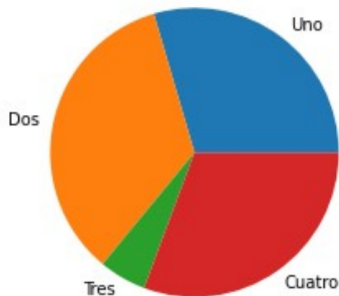
## Histogramas:



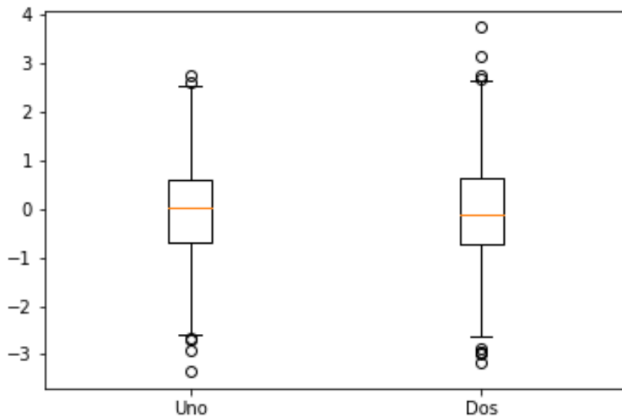
## Gráficos de barras:



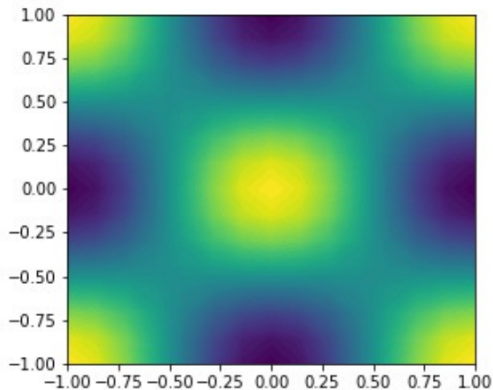
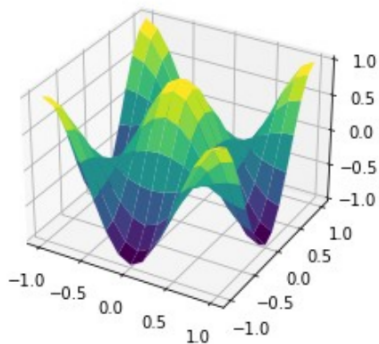
## Gráficos de sectores (quesos):



## Gráficos de cajas y bigotes:



## Gráficos de superficies y contornos:



# Matplotlib: Algunas funcionalidades

**Matplotlib** se encuentra muy bien integrada con otras bibliotecas como **NumPy**.

Por ejemplo, la biblioteca **Pandas** utiliza **Matplotlib** para realizar su propia visualización de datos.

Puedes comenzar a utilizar la biblioteca **Matplotlib** cargando en tu cuenta de GDrive el fichero **IntroMatplotlib.ipynb**.



- 6 La biblioteca NumPy
- 7 Visualización de datos con Matplotlib
- 8 Lectura de datos con Pandas**
- 9 Scikit-Learn

# Pandas

Pandas permite, de forma cómoda, cargar volúmenes de datos desde diferentes formatos y procesarlos, para extraer información relevante que posteriormente se use en la construcción de sistemas de IA y de Machine Learning.

Es una biblioteca básica en el área de las Ciencias de Datos.

Al igual que NumPy, es muy extensa.

En la asignatura usaremos **Pandas** de forma muy básica. Puedes encontrar una introducción más detallada en:

[https://pandas.pydata.org/docs/user\\_guide/10min.html](https://pandas.pydata.org/docs/user_guide/10min.html)



# Pandas: Tipos de datos

Tipos fundamentales sobre los que se construye la biblioteca:

- **DataFrame**: Tabla de datos
- **Series**: Conjunto de datos (columna)

Multitud de formatos de datos soportados: CSV, xls, HDF, SPSS, Pickle...

Al igual que las bibliotecas anteriores, se suele importar con alias desde Python:

```
1 import pandas as pd
```

# Pandas: Iniciación

En Pandas, los datos se cargan en tablas (**Dataframes**). Una vez leídos los datos en un dataframe `df`, algunos métodos útiles de Pandas son:

- **`df.head(n)`**: Muestra las `n` primeras filas del dataframe.
- **`df.tail(n)`**: Muestra las `n` últimas filas del dataframe `df`.
- **`df.sample(n)`**: Muestra `n` filas aleatorias del dataframe `df`.
- **`df.shape`**: Tupla conteniendo la forma (filas, columnas) del dataframe `df`.
- **`df.info()`**: Muestra un resumen de los tipos de datos de cada columna, y los valores perdidos/NaN que hay en el dataframe `df`.
- **`df.describe(n)`**: Muestra estadísticos básicos sobre las variables numéricas del dataframe `df` (media, desviación estándar, índices intercuartílicos. . . ).

Puedes comenzar a trabajar con **Pandas** incluyendo en tu GDrive el fichero **IntroPandas.ipynb**.



- 6 La biblioteca NumPy
- 7 Visualización de datos con Matplotlib
- 8 Lectura de datos con Pandas
- 9 Scikit-Learn

Scikit-learn (*SkLearn*) es una de las librerías de aprendizaje automático más populares y completas.



Se recomienda revisar y estudiar su documentación que, aparte de ser abundante, es muy útil:

[https://scikit-learn.org/stable/user\\_guide.html](https://scikit-learn.org/stable/user_guide.html)

La biblioteca **SkLearn** contiene multitud de datasets incluidos de modo que puedan utilizarse directamente para el estudio o la práctica de aprendizaje automático. Los más básicos se encuentran en el siguiente enlace:

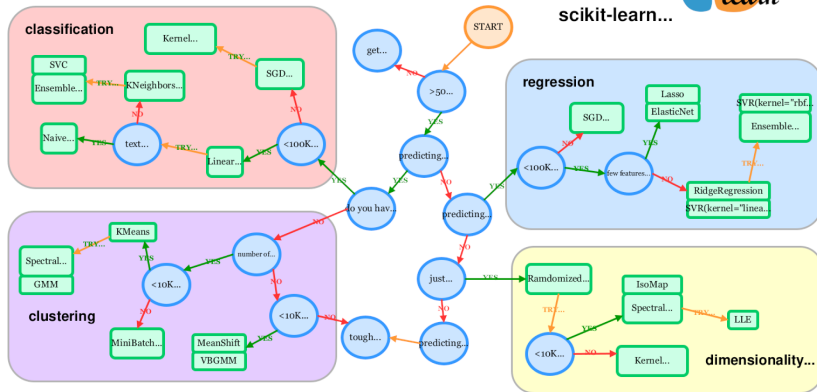
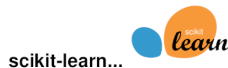
[https://scikit-learn.org/stable/datasets/toy\\_dataset.html](https://scikit-learn.org/stable/datasets/toy_dataset.html)

Un ejemplo de su uso:

```
1 from sklearn import datasets as ds
2
3 iris_data= ds.load_iris() # Carga del dataset de la flor
 de Iris
4
```

La biblioteca **SkLearn** está completamente integrada con **NumPy**.

La biblioteca **SkLearn** también contempla el uso de múltiples modelos para realizar aprendizaje:



Favorece el prototipado rápido. En pocas líneas podemos leer un conjunto de datos, entrenar un modelo y usarlo para predecir:

```
1 from sklearn import datasets as ds
2 from sklearn import svm
3
4 digits= ds.load_digits()
5 clasificador= svm.SVC(gamma=0.001, C=100.)
6 clasificador.fit(digits.data[:-1], digits.target[:-1])
7
8 pred= clasificador.predict(digits.data[-1:])
9 real= digits.target[-1]
10 print('Real: {}, Prediccion: {}'.format(real, pred[0]))
11
12 # Muestra 'Real: 8, Prediccion: 8'
13
```

Otro ejemplo: Regresión de datos.

```

1 from sklearn.linear_model import LinearRegression
2 import numpy as np
3
4 X= np.linspace(start=0, stop= 10, num=11)
5 y= 2*X-7
6
7 regresor= LinearRegression()
8 regresor.fit(X, y)
9 pred= regresor.predict([[11.0]])
10 # pred vale aproximadamente 15
11

```

Las funciones **fit** y **predict** se utilizan para entrenar el modelo y para ejecutarlo, respectivamente. Son comunes a los modelos incluidos en **SkLearn**.

# SkLearn: Algunas funcionalidades

Entre otras, ofrece las siguientes funcionalidades:

- **Particionamiento de datos** en entrenamiento y test.

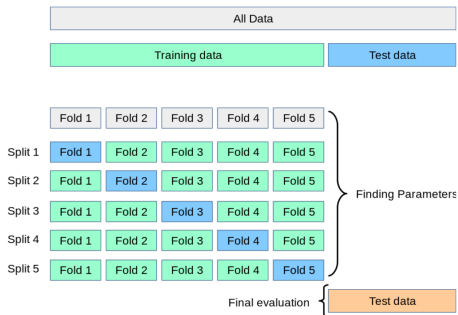
```
1 # 60% para training y 40% para test
2 X_train, X_test, y_train, y_test =
3 train_test_split(X, y, test_size=0.4)
```

- **Validación cruzada como protocolo de validación experimental:**

- **sklearn.model\_selection.Kfold:** División del conjunto de datos en K subconjuntos/folds/splits.
- **sklearn.model\_selection.StratifiedKFold:** Cada fold tiene el mismo porcentaje de muestras de cada tipo.
- **sklearn.model\_selection.ShuffleSplit:** Genera permutaciones aleatorias (no necesariamente disjuntas).

[https://scikit-learn.org/stable/modules/cross\\_validation.html](https://scikit-learn.org/stable/modules/cross_validation.html)

## Particionamiento vs. validación cruzada:



Se complementan. La validación cruzada se usa en el entrenamiento. La partición de test para validar el modelo entrenado.

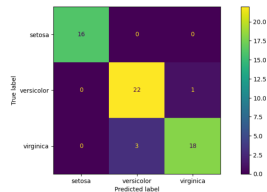
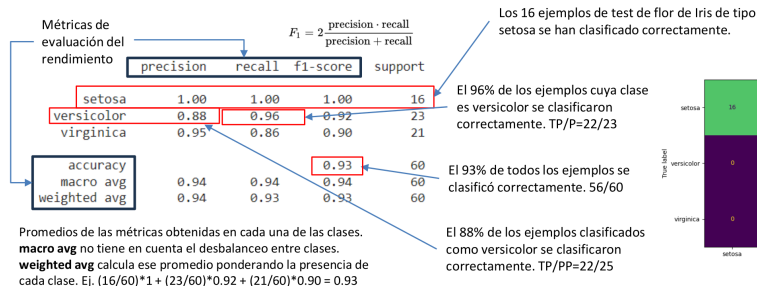
# SkLearn: Algunas funcionalidades

Más funcionalidades:

- **Evaluación de rendimiento** de modelos entrenados.

[https://scikit-learn.org/stable/modules/model\\_evaluation.html](https://scikit-learn.org/stable/modules/model_evaluation.html)

- `sklearn.model_selection.cross_val_score` o `cross_validate`
- `sklearn.metrics.confusion_matrix`
- `sklearn.metrics.classification_report`



# SkLearn: Algunas funcionalidades

Más funcionalidades:

- **Preprocesamiento de datos** (cambios de escala, codificación...).  
<https://scikit-learn.org/stable/modules/preprocessing.html>
- **Selección de características** (selección de atributos predictores).  
[https://scikit-learn.org/stable/modules/feature\\_selection.html](https://scikit-learn.org/stable/modules/feature_selection.html)
- **Tratamiento de valores perdidos** (mecanismos para tratar datos desconocidos en un conjunto).  
<https://scikit-learn.org/stable/modules/impute.html>
- **Optimización de hiperparámetros de modelos** (automatización de búsqueda de mejores condiciones de ejecución o aprendizaje de un modelo).  
[https://scikit-learn.org/stable/modules/grid\\_search.html](https://scikit-learn.org/stable/modules/grid_search.html)
- **Encadenamiento/composición de operaciones** (Pipelines).  
<https://scikit-learn.org/stable/modules/compose.html>

Puedes comenzar a trabajar con **SkLearn** incluyendo en tu GDrive el fichero **IntroSkLearn.ipynb**.



# Aprendizaje Automático

## Introducción a las prácticas

Manuel Pegalajar Cuéllar<sup>1</sup>

<sup>1</sup>Departamento de Ciencias de la Computación e Inteligencia Artificial. Universidad de Granada